

Projet Laravel + Angular : Gestion de Pressing / Laverie

Description du projet

Le pressing **LIC** souhaite développer une application web de gestion des commandes de nettoyage, réalisée avec une API Laravel et un frontend Angular. L'objectif est de dématérialiser le dépôt des commandes, le suivi de leur traitement et l'encaissement, tout en informant automatiquement le client par email.

Acteurs et rôles

- Gestionnaire : gère les services, les tickets, les statuts, les paiements et consulte les statistiques.
- Client : consulte les services, dépose une commande (ticket), suit son statut et consulte l'historique de ses commandes.

Fonctionnalités

1. Gestion des services

Le gestionnaire peut ajouter, modifier, archiver ou supprimer un service.

- Chaque service contient : libellé (ex : lavage, repassage, nettoyage à sec), prix unitaire, description, disponibilité (actif/inactif).
- Un service désactivé n'apparaît plus dans le catalogue client et ne peut plus être sélectionné pour une nouvelle commande.

2. Gestion des commandes (tickets)

Un client peut :

- Consulter le catalogue des services (avec filtre par libellé).
- Déposer une commande en sélectionnant un ou plusieurs services, chacun avec une quantité (ex : 3 pantalons à repasser, 2 kg de lavage).
- Suivre le statut de sa commande.
- Consulter la liste et le détail de ses commandes passées, avec le reçu correspondant.

Le gestionnaire peut :

- Lister l'ensemble des tickets et en voir le détail.
- Annuler un ticket (avant qu'il soit prêt).
- Faire évoluer le statut d'un ticket selon le cycle suivant :

Statut	Déclencheur / Rôle	Effet
Reçu	Automatique à la création de la commande (client)	Email de confirmation au client + notification au gestionnaire
En traitement	Gestionnaire	Le ticket est en cours de lavage/repassage
Prêt	Gestionnaire	Email au client + génération du reçu PDF
Récupéré	Gestionnaire (au retrait, avec enregistrement du paiement si non payé)	Clôture du ticket

3. Gestion des paiements

- Paiement en espèces enregistré par le gestionnaire, au dépôt ou au retrait du ticket.
- Un ticket ne peut être payé qu'une seule fois ; le montant et la date du paiement sont enregistrés.
- Un ticket ne peut passer au statut « Récupéré » que s'il est payé.

4. Authentification et rôles

- Inscription / connexion des clients.
- Compte gestionnaire créé en base (pas d'auto-inscription).
- Accès aux routes API protégé selon le rôle (Laravel Sanctum recommandé pour l'API Angular).

5. Statistiques et rapports (Chart.js côté Angular)

- Nombre de tickets créés le jour même.
- Nombre de tickets récupérés / clôturés le jour même.
- Recette journalière (total des paiements encaissés).
- Nombre de tickets par mois (graphique).
- Répartition du chiffre d'affaires par service, par mois (graphique).

6. Notifications et automatisation

- Email de confirmation envoyé au client après le dépôt d'une commande.
- Notification au gestionnaire lors de la création d'une nouvelle commande.
- Reçu / facture PDF généré et envoyé par email au client lorsque le ticket passe au statut « Prêt ».

Architecture technique attendue

- Backend : Laravel (API REST), authentification par token (Sanctum).
- Frontend : Angular (SPA consommant l'API).
- Base de données : MySQL ou PostgreSQL.
- Génération PDF : DomPDF ou équivalent côté Laravel.
- Envoi d'emails : Laravel Mail (Mailtrap ou équivalent en environnement de test).

DevOps

Mettre en place une chaîne d'intégration continue simple (GitHub Actions recommandé) réalisant :

- Le dépôt du projet sur GitHub, sur une branche nommée nom_prenom_pressing.
- L'installation des dépendances (composer install côté API, npm install côté Angular).
- L'exécution des tests automatisés existants.
- (Bonus) La construction d'une image Docker pour l'API et/ou le frontend.

Livrables attendus

- Code source du backend (Laravel) et du frontend (Angular) sur un dépôt GitHub.
- Fichier de configuration CI (GitHub Actions / GitLab CI / Jenkinsfile).
- Jeu de données de test (seeders) : quelques services et tickets d'exemple.
- Courte documentation (README) expliquant l'installation et le lancement du projet.

Évaluation du projet

- **Projet : 14 points**
- **Entretien individuel (One-to-One) avec le professeur : 6 points**

Total : 20 points

L'entretien individuel permettra à chaque étudiant de présenter et d'expliquer son travail, de justifier ses choix techniques et de répondre aux questions du professeur sur le projet réalisé.